



***Final year project submitted in partial fulfillment of the requirements
For the Degree of Bachelor of Telecommunication Engineering***

***Google Assistant & GSM-Based Home Automation
System (IoT)***

Supervised By

Engr. Mujeeb ur Rehman

Submitted By:

Group Leader

Ibadullah (22TC-45)

Group Members

Muhammad Jaffar (22TC-29)

Aijaz Ahmed Ghoto (22-21TC22)

***Department of Telecommunication Engineering
Quaid-e-Awam University of Engineering, Science & Technology
Nawabshah***

2026



Quaid-e-Awam University of Engineering, Science, and Technology, Nawabshah

Department of Telecommunication Engineering

Performa for Final Year Project Proposal

Group No. 06

Batch: 22TC

Group Member		Title of the Final Year Project						SDGs by 2030	
1.	22TC45	Google Assistant & GSM Based Home Automation System (IoT)						 Please check the information of SDGs on https://www.sdgpakistan.pk/	
2.	22-21TC22								
3.	22TC29								
Target SDGs (√) Min 1-2 SDGs		1.		7.		13.			
		2.		8.		14.			
		3.		9.	✓	15.			
		4.	✓	10.		16.			
		5.		11.		17.			
		6.		12.					
Description of SDG		SDG 4 – Quality Education: Provides practical learning and technical skills through a real-world engineering project. SDG 9 – Industry, Innovation and Infrastructure: Promotes innovation by developing a low-cost home automation system using modern IoT and GSM technology.							
Aim and Objectives									
Aim		To develop a low-cost hybrid home automation system enabling remote control of household appliances through Google Assistant & GSM-based SMS.							
Objectives		- To control lights and fans using Google Assistant through IoT technology. - To control appliances through GSM-based SMS during internet failure. - Integrate Arduino Uno, ESP32, SIM800L, and relay modules into system. - To provide reliable online and offline control of household appliances.							
Starting Date				Date of completion					
Academic Supervisor				Engr. Mujeeb ur Rehman Lecturer, Telecommunication Engineering Department, QUEST, Nawabshah				Signature	
Industrial Collaborator /Co-Supervisor if any				Engr. Sadullah Mangrio TAC Engineer, Cyber Internet Services					
Approved by FYP Committee									

DECLARATION

This thesis designated as "Google Assistant and GSM-Based Smart Home Automation System" is supervised by Engr. Mujeeb ur Rehman of the Telecommunication Engineering Department at Quaid-e-Awam University of Engineering, Sciences & Technology (QUEST), Nawabshah.

We declare that the information written in this thesis book is the result of our own efforts and hard work. This thesis book has not already been used for any other degree or diploma in any other institute. We will submit this thesis book as a Bachelor Final Year Group Project.

Ibadullah (22TC-45)

Muhammad Jaffar (22TC-29)

Aijaz Ahmed Ghoto (22-21TC22)

DEDICATION

We never dreamed about success but worked for it with self-effort and guidance from respected faculty members. Our humble efforts are dedicated to those very close to our hearts.

OUR RESPECTABLE AND HONORABLE ELDERS

FATHER AND MOTHER

Whose encouragement, efforts, love, and prayers of day and night enable us to achieve such success and honor.

Quaid-e-Awam University of Engineering, Science & Technology Nawabshah
Department of Telecommunication Engineering



CERTIFICATE

This is to certify that the following students of the 22TC batch have completed the final year project in fulfillment of the requirements for the degree of Bachelor of Engineering in Telecommunication Engineering from Quaid-e-Awam University of Engineering, Science & Technology, Nawabshah, Sindh, Pakistan. The thesis titled **“Google Assistant & GSM Based Home Automation System (IoT)”** is submitted to Quaid-e-Awam University of Engineering, Science & Technology, Nawabshah, Sindh, Pakistan for the award of the degree of Bachelor of Engineering.

Ibadullah	22TC-45	(GL)
Aijaz Ahmed Ghoto	22-21TC-22	(AGL)
Muhammad Jaffar	22TC-29	

SUPERVISOR

Engr. Mujeeb ur Rehman
Department of Telecommunication
Engineering
Quaid-e-Awam University of Engineering,
Science &
Technology Nawabshah

CHAIRMAN

Prof. Dr. M. Mujtaba Shaikh
Department of Telecommunication
Engineering
Quaid-e-Awam University of Engineering,
Science &
Technology Nawabshah

CO SUPERVISOR

Engr. Sadullah Mangrio
TAC Engineer
Cyber Internet Services

EXTERNAL

Dr. Waheeduddin Hyder
MUET, Jamshoro

Acknowledgement

All praise and gratitude are due, first and foremost, for the strength, patience, and clarity of thought granted throughout the course of this project. This work would not have been possible without the guidance, encouragement, and technical support of many individuals.

We would like to express our sincere and heartfelt gratitude to our supervisor, Engr. Mujeeb ur Rehman, for the invaluable guidance, constructive criticism, and continuous encouragement provided throughout every stage of this project, from the initial concept to the final testing of the Google Assistant and GSM-Based Smart Home Automation System.

We are also thankful to the faculty members of the Telecommunication Engineering Department for providing the laboratory facilities, technical resources, and academic environment necessary to design, assemble, and test the hardware prototype presented in this thesis.

We are grateful to our families and friends who have been a constant source of encouragement to us throughout our experience.

Ibadullah

Abstract

In this research, the disadvantages of classical home automation solutions that depend only on Wi-Fi connections and internet-based voice assistants are examined. Indeed, the lack of availability of Wi-Fi connections and voice assistants during power outages and router malfunctions poses a real problem in the context of controlling household appliances. The absence of stable internet and electricity makes it more difficult to manage household devices remotely. In cases where the home router becomes inoperative, all network devices go offline simultaneously. A double-mode smart home automation system has been designed to address the issue. The system combines the cloud-based voice control mechanism with an independent GSM/SMS mechanism of control.

A Google Assistant and GSM-based smart home automation system has been designed and implemented in the course of this thesis. ESP32 microcontroller allows connecting to the SinricPro cloud platform to control household appliances using Google Home and Google Assistant. At the same time, SIM800L GSM module is connected to Arduino Uno for SMS-based control independent of any internet connection. The four-channel relay module is used to control AC loads such as a cooling fan, a plug socket, and two light bulbs. The power module employs 9 V, 2 A power source, while the power needed for SIM800L module is provided by LM2596 buck converter. Voltage divider maintains the signal entering ESP32 RX pin at a safe level.

According to empirical testing, voice commands sent through Google Home led to the reaction of devices with the constant delay of one second in the case of using the SinricPro route of control. SMS commands received by SIM800L led to successful control of the appliances in the offline mode with the delay of approximately three seconds. Several tests and evaluations have confirmed that both control mechanisms may work independently as alternative modes of control of the connected devices without any impact on their performance. Therefore, voice control IoT solutions coupled with the independent GSM/SMS solution represent a good and affordable option for home automation. The project involves easy-to-use components that make it suitable for educational purposes.

Keywords: *Application of smart home automation technology with ESP32, SinricPro, Google Assistant and Google Home, including implementation of a SIM800L GSM module and Arduino Uno, using IoT. The system uses relay module functions and offline control features, within the domain of embedded systems.*

Table of Contents

Certificate	4
Acknowledgement	5
Abstract.....	6
Table of Contents	8
List of Figures	11
List of Tables	12
List of Abbreviations	13
Chapter 1: Introduction.....	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Statement.....	2
1.4 Objectives.....	2
1.5 Scope of the Project	3
1.6 Significance of the Study	3
1.7 Organization of the Thesis	3
Chapter 2: Literature Review	4
2.1 Overview	4
2.2 GSM-Based Home Automation Systems.....	4
2.3 Voice-Assistant-Controlled IoT Systems (Google Assistant, Google Home)	4
2.4 Cloud-Based Device Control Platforms (SinricPro).....	5
2.5 ESP32-Based Home Automation	6
2.6 Offline/SMS-Based Control Systems (SIM800L).....	6
2.7 Research Gap.....	7
2.8 Summary of Chapter	7
Chapter 3: Methodology	8
3.1 System Overview	8
3.2 System Block Diagram	8
3.3 Working Principle.....	9
Stage 1 – Voice Command via Google Assistant / Google Home, SinricPro Cloud.....	9

Stage 2 – ESP32 Receiving SinricPro Command and Driving Relays	9
Stage 3 – Manual/Offline Control via SIM800L SMS Commands	9
Stage 4 – Arduino Uno as Central Controller Coordinating ESP32 & SIM800L	9
Stage 5 – Power Stage (9V 2A Adapter → LM2596 Buck to 4V for SIM800L) and Voltage Divider on ESP32 RX.....	9
Stage 6 – Status Feedback / Confirmation (SMS or Serial Feedback).....	10
3.4 System Flowchart	10
3.5 Design Considerations	11
Chapter 4: Hardware Components and Circuit Design	12
4.1 Introduction	12
4.2 Component Overview	12
4.3 Component Description	13
(a) Arduino Uno	13
(b) ESP32 Module	14
(c) SIM800L GSM Module.....	14
(d) 4-Channel Relay Module.....	15
(e) LM2596 Buck Converter.....	16
(f) SMPS Power Supply Module.....	16
(g) Voltage Divider Circuit (RX Protection).....	16
(h) Cooling Fan, AC Plug Socket, and Light Bulbs (Output Loads).....	17
4.4 Schematic Diagram.....	17
4.5 Pin Configuration Summary.....	18
4.6 Power Supply Design.....	19
4.7 Communication Interface.....	19
Chapter 5: Software Implementation.....	20
5.1 Overview	20
5.2 Arduino Uno Firmware.....	20
5.3 ESP32 Companion Firmware.....	20
5.4 SinricPro and Google Home/Google Assistant Configuration.....	21
5.5 GSM/SMS Command Protocol via SIM800L.....	21

5.6 Serial Communication Protocol between Modules	21
5.7 Error Handling and Robustness.....	21
5.8 Memory and Resource Considerations	22
Chapter 6: Results and Discussion	23
6.1 Testing Methodology	23
6.2 Voice Command Response Test (Google Assistant/Google Home Appliance via SinricPro)	23
6.3 SMS/Offline Control Test via SIM800L.....	24
6.4 Response Time and Reliability Measurement	24
6.5 Power Consumption Test	24
6.6 Discussion of Results.....	25
6.7 Prototype Photographs (Results)	26
6.8 Limitations Observed During Testing	29
Chapter 7: Conclusion and Future Work.....	30
7.1 Conclusion.....	30
7.2 Future Work	31
References.....	32
Appendix A: Complete Firmware Source Code	32
A.1 Arduino Uno Firmware (SIM800L SMS Control)	34
A.2 ESP32 Firmware (SinricPro / Google Home Control).....	38
Appendix B: Bill of Materials.....	42
Appendix C: User Manual / Operating Instructions.....	43
C.1 Powering On the System	43
C.2 Controlling Appliances via Google Assistant / Google Home	43
C.3 Controlling Appliances via SMS (Offline Mode).....	43
C.4 Troubleshooting	44

List of Figures

Figure 3.1: System Block Diagram of the Smart Home Automation System

Figure 3.2: Operational Flowchart of the Voice-Command and SMS-Command Paths

Figure 4.1: Overview of Major Hardware Components Used in the System

Figure 4.2: Arduino Uno Picture

Figure 4.3: ESP32 Module Picture

Figure 4.4: SIM800L GSM Module Picture

Figure 4.5: 4-Channel Relay Module Picture

Figure 4.6: LM2596 Buck Converter Picture

Figure 4.7: SMPS Power Supply Module Picture

Figure 4.8: Schematic Wiring Diagram of the Complete System

Figure 6.1: Complete Assembled Prototype (Photograph)

Figure 6.2: Close-up of Breadboard Wiring (ESP32 and SIM800L)

Figure 6.3: Screenshot of Google Home App Device Control

Figure 6.4: Screenshot of Received SMS Confirmation

List of Tables

Table 4.1: Detailed Description of Hardware Components

Table 4.2: Pin Configuration Summary

Table 5.1: SMS Command Protocol Fields

Table 6.1: Voice Command Response Time Test Results

Table 6.2: SMS Command Response Time Test Results

Table 6.3: Estimated Power Consumption Under Different Operating States

List of Abbreviations

AC	Alternating Current
DC	Direct Current
ESP32	Espressif Systems 32-bit Wi-Fi/Bluetooth Microcontroller
GSM	Global System for Mobile Communications
GPIO	General Purpose Input/Output
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
LED	Light Emitting Diode
MCU	Microcontroller Unit
PWM	Pulse Width Modulation
RX/TX	Receive / Transmit (serial lines)
SMS	Short Message Service
SMPS	Switched-Mode Power Supply
SSID	Service Set Identifier (Wi-Fi network name)
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
Wi-Fi	Wireless Fidelity (IEEE 802.11)

Chapter 1: Introduction

1.1 Background

Smart-home appliances are one of the emerging areas of application of the Internet of Things (IoT) technology and embedded systems where household devices can be controlled from smartphones and even voice assistants. The voice assistant and ecosystem such as Google Assistant and Google Home have made it possible for the ordinary people to enjoy voice-controlled appliance control in their homes.

However, most existing consumer-level smart home solutions work solely dependent on a reliable Wi-Fi network or internet connection. For places which experience power cuts, router crashes or even disruptions in internet connection services, it becomes very hard to use Wi-Fi-dependent automation and control of appliances when they become really necessary. It has created a situation where users can benefit from the conveniences of voice-controlled appliances only if they have access to reliable Wi-Fi networks.

The current project tries to fill this gap in the market where voice-controlled pathway through the cloud is merged with another independent pathway of controlling devices through SMS or GSM connection. This will make it possible for the user to control household devices using SMS in case the Wi-Fi network is not accessible to him. Instead of having the offline control pathway as the less important pathway, it has been incorporated into the system so that both pathways converge on a common relay-driving controller thus making them both equally reliable in activating all the connected devices.

Pakistan and other similar countries frequently experience the interruptions in internet connectivity due to power cuts and the same applies to home Wi-Fi routers despite the existence of continuous GSM connectivity in those places.

1.2 Motivation

Motivation for the project comes from the need to design a state-of-the-art smart home automation system that utilizes voice control as well as cloud connectivity, but is able to function without internet connection. Involving the use of ESP32 microcontroller that allows interaction with SinricPro cloud for making the system compatible with Google Assistant and Google Home, together with SIM800L GSM module paired with Arduino Uno for SMS-based control in an offline mode, this project proves that a budget-friendly automation solution based on dual paths may be developed using available parts.

Another aspect of the motivation is related to educational purposes since using both cloud-connected IoT path and traditional embedded GSM communication path in the project provides practical experience in a wide variety of telecommunications engineering concepts. These include low-level serial communication, power regulation, cloud API integration and voice command processing, thus allowing for the consolidation of theoretical knowledge into a testable practical implementation. Solving practical problems, such as providing proper power supply to a GSM module, protecting logic levels of different devices with varied voltage tolerance and coordinating two microcontrollers in one system with common outputs, gave the opportunity to deal with issues that arise when designing modern systems.

Finally, another motivation for developing the project comes from the need to create something usable, not just fulfilling degree requirements. Thus, having designed a prototype which would be used in the future by team members and their families, the motivation to address practical reliability issues, such as the GSM power spike mentioned in Section 2.6, emerged.

1.3 Problem Statement

The existing smart home devices that utilize only Wi-Fi technology become completely unusable in case of internet failure, which makes it impossible for users to have remote appliance control. It is necessary to have a cost-effective system of home automation that will provide voice control capabilities when there is access to the internet, and a reliable alternative when the internet is not accessible.

To be more precise, this problem can be broken down into the following three sub-problems, which are solved by this project: (1) having a voice control capability for household appliances through a well-known, free assistant system; (2) providing an equal offline capability that will not depend on any internet technology; and (3) having two pathways that will work with the same relay device without conflicts. A system that solved only the first two sub-problems, without addressing the third, would still be prone to unpredictable behaviour whenever both paths were used close together in time, which is why the master-controller architecture described in Chapters 3 and 5 was treated as a core design requirement rather than an optional refinement.

1.4 Objectives

- To design and implement an ESP32-based smart home controller that integrates with Google Assistant and Google Home through the SinricPro cloud platform.
- To implement an independent, offline SMS-based control path using a SIM800L GSM module interfaced with an Arduino Uno.
- To control multiple AC appliances (fan, plug socket, and lighting) using a 4-channel relay module driven by the microcontrollers.
- To design a stable power supply system using an SMPS source and an LM2596 buck converter to safely power the SIM800L module.
- To protect the ESP32 RX pin using a voltage divider circuit for reliable UART communication.
- To evaluate the response time and reliability of both the voice-command and SMS-command control paths through practical testing.

1.5 Scope of the Project

This project involves designing, assembling hardware, developing firmware, and carrying out functional tests on a smart home automation system prototype. This system is able to control a certain number of household appliances, which are a fan, an electrical outlet, and two lamps, from a single control panel. This project will be carried out under the supervision of an indoor controlled environment. Suggestions for the future include deployment in multiple rooms and inclusion of more appliances.

The scope of this project does not involve certification to meet electrical safety standards, development of a weather-resistant housing, or linking up with other voice assistant ecosystems apart from Google Assistant / Google Home. These are some of the necessary stages in order for this to be ready for commercial or permanent household use.

1.6 Significance of the Study

This project offers a practical implementation of two-channel home automation by incorporating the ease of access to voice-based cloud assistants with the stability of GSM-based offline control system. This project serves as a basis for those who wish to build low-cost and stable home automation projects which can run even during internet outages. Because all information about the firmware code, circuit diagram, and bill of material has been included in this thesis (Chapter 4-5 and Appendix A-C), it becomes possible to utilize this project as a basis for future projects for other student groups with little extra effort.

1.7 Organization of the Thesis

The thesis is subdivided into seven chapters. Chapter one deals with the background, rationale, objectives, and scope of the research. Chapter two is a literature review on previous work in the field. Chapter three contains the system methodology. Chapter four contains the hardware part and the circuit diagrams. Chapter five covers the software part. Chapter six contains the results of testing.

Chapter 2: Literature Review

2.1 Overview

The objective of this chapter is to review the research literature and technology regarding home automation, giving special focus to automation systems that use GSM platform, Voice Control IoT devices, Cloud based internet control of devices, ESP32 automation, and SMS-based offline automation.

2.2 GSM-Based Home Automation Systems

GSM has been an offline dependable solution for home automation because SMS can be sent and received in areas covered by cellular networks and even not having Wi-Fi or broadband at home. There were numerous examples in previous researches showing that GSM modules SIM800/SIM900 can be used for controlling relays using SMS messages. This is a low-cost offline method of home devices management.

Conventional GSM home automation systems are comprised of AT-command compatible modules which communicate with microcontroller using the UART interface. Microcontroller analyses SMS text and matches keywords to relay outputs. The advantages of such system are cost effectiveness – there is no need for subscription and permanent internet connection. This method works with any GSM module and minor changes of firmware. As the disadvantages of traditional methods, it was mentioned long SMS delay and good cellular connection requirement.

In student or hobbyist researches, the problem of power supply design in GSM-based projects has been often described. There is a fact that GSM module can consume much more current on message registration and message sending in comparison with normal work of the device. Circuits, which connect GSM module and microcontroller using common, weakly regulated power rail, have problems with message registration and sending.

2.3 Voice-Assistant-Controlled IoT Systems (Google Assistant, Google Home)

Voice controls for a smart home using Google Assistant and Google Home are extremely easy to implement because they transfer the spoken word to the cloud and, therefore, both the smartphone or the speaker and the smart home hub should have access to the Internet.

It is also true about similar systems such as Amazon Alexa and Apple HomeKit because voice commands are recognized and translated into actions by the smart devices in the cloud. In this project, Google Home/Assistant was selected because it is popular, provides a free developer program for smart homes, and supports third-party bridges like SinricPro.

The advantage of voice assistants is their simplicity: no application needs to be downloaded or any specific commands to be used; for instance, "turn on the fan" is a single command within ON/OFF. The biggest problem with voice assistants highlighted in home automation literature is that they cannot be used when there is no access to the Internet.

2.4 Cloud-Based Device Control Platforms (SinricPro)

SinricPro is one such example where cloud platforms are used for connecting microcontroller IoT devices with larger voice assistants. The cloud platform takes care of discovering the device, signing into it, and sending command instructions. This significantly reduces the development effort needed to expose ESP32-based devices to Google Home and Google Assistant.

Compared to building a custom MQTT broker and a separate Google Smart Home Action from scratch, using an intermediary platform such as SinricPro allows a student project to reach a working voice-control demonstration in a fraction of the development time, since the platform already implements OAuth-based account linking, device-state synchronization, and the Google Smart Home fulfillment API on the developer's behalf. The trade-off is a dependency on a third-party cloud service remaining online and within its free-tier usage limits.

For a project of this scale, the free-tier device and command limits offered by SinricPro were more than sufficient, since only four switch devices were required and the volume of voice commands generated during development, testing, and demonstration remained well within the platform's free usage allowance throughout the project timeline.

2.5 ESP32-Based Home Automation

The ESP32 microcontroller, with its integrated Wi-Fi and Bluetooth radios, dual-core processor, and rich set of GPIO pins, has become a popular platform for IoT and home automation projects. Its ability to run cloud SDKs like the SinricPro library directly on the device makes it ideal for small, single-board smart home controllers.

Unlike older Wi-Fi microcontrollers such as the ESP8266, the ESP32's dual-core design allows network and cloud tasks, along with local I/O operations, to run without blocking delays. Its larger flash and RAM space can easily handle TLS-secured cloud SDKs like SinricPro, along with application logic. These features are the main reasons the ESP32 was chosen as the voice control microcontroller for this project instead of an ESP8266 or a simple Wi-Fi shield.

The wide availability of ESP32 development boards, plenty of community documentation, and built-in compatibility with the Arduino IDE made development easier for a student project team working on a tight semester schedule. The same familiar Arduino-style programming model used for the Arduino Uno firmware could be reused for the ESP32 firmware with only minor syntax changes.

2.6 Offline/SMS-Based Control Systems (SIM800L)

The SIM800L is a small, affordable GSM/GPRS module often used in embedded projects for SMS communication. Its AT-command interface lets a host microcontroller, like an Arduino Uno, send and read SMS messages. This enables straightforward text-based command protocols for remote control applications that do not require internet access.

A common issue noted by hobbyists and mentioned in module datasheets is the SIM800L's current draw during transmission bursts. This can exceed 2A for short periods, which may cause voltage drops on poorly regulated supplies and lead to unexpected resets. This project tackles that problem with a dedicated LM2596 buck-converter rail, as explained in Chapter 4.

The SIM800L also requires a valid, network-registered SIM card and adequate antenna placement to reliably send and receive SMS messages, both of which were verified during the hardware bring-up phase of this project before any firmware development began, to isolate GSM-connectivity issues from software issues during subsequent debugging.

2.7 Research Gap

While GSM-based control and cloud voice-assistant control have each been explored independently in prior literature and hobbyist projects, relatively few low-cost prototypes integrate both into a single system where the offline SMS path acts as a genuine fallback for the cloud voice path, sharing the same relay outputs. This thesis aims to fill that gap by presenting an integrated, dual-path smart home automation system.

Specifically, published hobbyist and academic project reports reviewed for this thesis tended to treat GSM/SMS control and cloud/voice control as alternative, mutually exclusive design choices rather than complementary layers of the same system; where dual-path designs were mentioned, implementation details on how conflicting commands from two independent microcontrollers sharing the same relay hardware were avoided were rarely discussed in depth, which is a specific gap this thesis addresses through the master-controller architecture presented in Chapter 3.

2.8 Summary of Chapter

This chapter reviewed literature relevant to GSM-based home automation, voice-assistant-controlled IoT systems, cloud-based device control platforms, ESP32-based automation, and offline SMS control. The reviewed studies provide the technical foundation for the dual-path methodology presented in the next chapter.

Chapter 3: Methodology

3.1 System Overview

The proposed system is composed of two cooperating control paths that both ultimately drive the same 4-channel relay module: a cloud/voice path built around the ESP32 and SinricPro, and an offline SMS path built around the Arduino Uno and SIM800L module. Either path can independently switch the connected appliances, giving the user redundant means of control.

From a design perspective, the two paths were deliberately kept as loosely coupled as possible above the relay-driving layer: the ESP32 has no knowledge of SMS command syntax, and the SIM800L/Arduino side has no knowledge of SinricPro device IDs or cloud protocols. The only point of integration between the two paths is the short, clear serial command interface described in Section 3.3. This design allows each path to be tested independently. It also lowers the chance that a bug in one path will impact the other.

3.2 System Block Diagram

The block diagram for the proposed system is shown in Figure 3.1, which represents the flow of data between the voice command line, SMS command line, and output of relay / appliance.

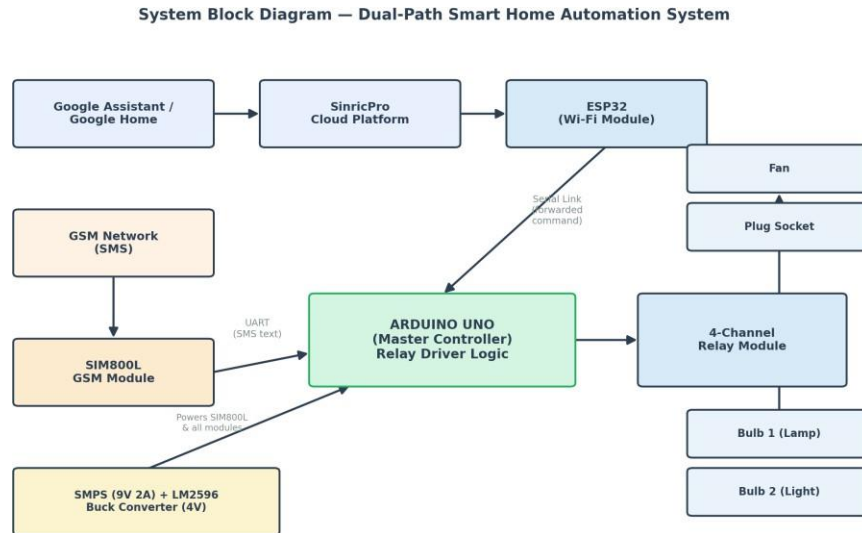


Figure 3.1: System Block Diagram of the Smart Home Automation System

3.3 Working Principle

The system operates through the following stages:

Stage 1 – Voice Command using Google Assistant / Google Home to SinricPro Cloud

The user delivers an order using either Google Assistant or Google Home application (e.g., “turn on the fan”). Google Home sends this order to the SinricPro cloud system, which recognizes the corresponding ESP32 unit and sends the command to turn on the switch. All this process is executed via an established secure connection that exists between the SinricPro cloud service and the ESP32 due to the presence of the SinricPro SDK running on ESP32. Thus, there is no need for any additional authentication process right at this point.

Stage 2 – ESP32 Receiving SinricPro Order and Relay Triggering

ESP32 stays connected all the time with the SinricPro cloud over Wi-Fi and gets the order from the SinricPro via a callback function and triggers the corresponding relay channel of the four-channel relay module in order to turn the device ON/OFF. Being a master-controller device in this project implementation, ESP32 does not control the relay itself directly and simply sends a short order in the form of the text message to the Arduino Uno, which controls the relay triggering, see Stage 4 below.

Stage 3 – Manual/Offline Control by SIM800L SMS Orders

Apart from the voice pathway described above, the user is also able to send the text SMS order (e.g., “fanon”) to the SIM800L SIM number. SIM800L will receive the order and will send it further to the Arduino Uno via UART. This pathway functions offline as it uses only the GSM signal without any Internet connectivity, thus demonstrating higher resistance to power failures and broadband outage problems compared to the pathway described above.

Stage 4 – Arduino Uno as the Central Controller: Direct Control of ESP32 and SIM800L

In this stage, Arduino Uno is responsible for decoding the received SMS commands based on the predefined list of commands and activating the respective relay channel directly without passing through the path ESP32/SinricPro. It should be noted that such solution guarantees SMS functionality regardless of the availability of the network. As the Arduino Uno is the only controller that is directly connected to the relay inputs, the SMS path and voice command path are merged into one command decoding procedure, which excludes the conflicts between paths when it comes to relays operation.

Stage 5 – Power Stage (9 V, 2 A Adapter; LM2596 Buck Converter to 4 V for SIM800L) and ESP32 RX Voltage Divider

For providing the power, the SMPS module is used that is supplied from 9 V, 2 A adapter. This solution is needed due to the requirement of the SIM800L to have a steady power supply from 3.7 to 4.2 V with the ability to provide the peak current when it operates in GSM mode. For this reason, the LM2596 buck converter provides 4 V supply for SIM800L. The voltage divider is used for dividing the voltage level of ESP32 RX line from 5V TTL to 3.3V.

Stage 6 – Status Feedback and Confirmation (SMS or Serial Feedback)

After the completion of command execution, the confirmation SMS is sent to the sender, and at the same time, ESP32 sends an update regarding the device status to the SinricPro Cloud.

3.4 System Flowchart

Figure 3.2 shows the flowchart of system operation, indicating the decision-making process for voice command method and SMS command method.

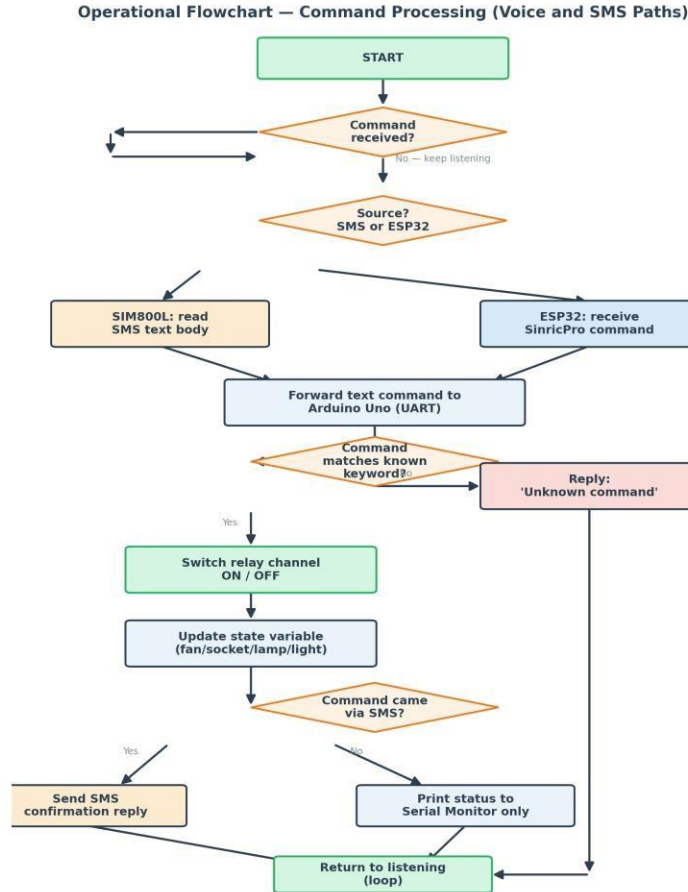


Figure 3.2: Operational Flowchart of the Voice-Command and SMS-Command Paths

3.5 Design Considerations

- There are two separate control channels to make sure that control of the appliance remains available even if there is a failure of the internet or Wi-Fi.
- In terms of safety, the ESP32 RX pin is protected by a voltage divider and the SIM800L is supplied with a stable and isolated power source by a dedicated buck converter.
- The system features modularity in that voice control, SMS control, and relay switching are each contained within separate functional blocks which are linked together via clear interfaces.
- It is low in cost since all the components are inexpensive modules that are readily available and suitable for use within a student's prototype budget.

Chapter 4: Hardware Components and Circuit Design

4.1 Introduction

The chapter gives a detailed account of each of the hardware components used in the building of the Google Assistant and GSM-Based Smart Home Automation System, together with the full circuit interconnection diagram and an explanation of the electrical design and power distribution strategy.

The hardware design decisions explained in this chapter were mainly based on the three factors mentioned in Chapter 3: ensuring that the two control paths remained electrically and logically separate above the relay level, providing the SIM800L with a power supply that could cope with its transmission current spikes, and guarding the ESP32's logic pins which operate at 3.3V against any accidental contact with 5V signals found elsewhere on the board.

4.2 Component Overview

An overview of the major hardware components used in the system is given in Figure 4.1.

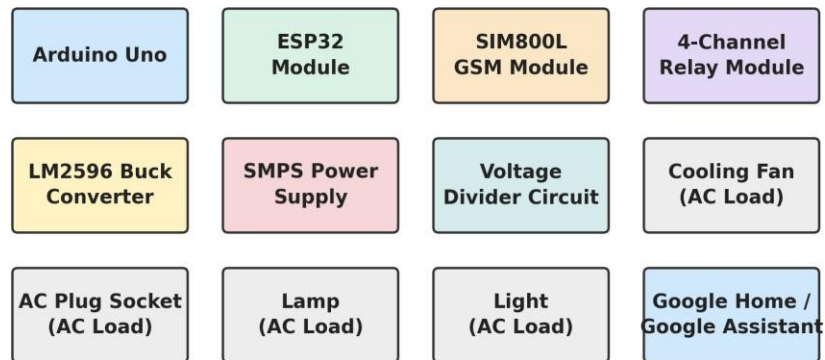


Figure 4.1: Overview of Major Hardware Components Used in the System

Component	Function in the System	Key Specifications
Arduino Uno	Reads and parses SMS commands from the SIM800L and drives the relay module for offline control.	ATmega328P; 14 digital I/O; 5V DC
ESP32	Connects to Wi-Fi and the SinricPro cloud; receives voice commands and drives the relay module.	Dual-core; Wi-Fi/BT; 3.3V logic

Component	Function in the System	Key Specifications
SIM800L GSM Module	Sends/receives SMS commands over the cellular network for offline appliance control.	Quad-band GSM/GPRS; 3.7–4.2V
4-Channel Relay Module	Switches AC loads (fan, socket, bulbs) based on commands from the Arduino/ESP32.	5V trigger; up to 10A per channel
LM2596 Buck Converter	Steps down the 9V supply to a stable 4V regulated output for the SIM800L.	Adjustable buck; up to 3A output
SMPS Power Supply	Provides the primary 9V 2A DC supply for the entire system from AC mains.	AC-DC; 9V 2A output
Voltage Divider Circuit	Steps down 5V logic on the RX line to a safe level for the ESP32's 3.3V input.	Resistor divider network
Cooling Fan (Output Load)	AC appliance load switched by one relay channel.	AC-powered fan
AC Plug Socket (Output Load)	Switched AC socket for connecting general-purpose appliances.	Standard AC socket
Light Bulbs (2x, Output Loads)	AC lighting loads, each switched by a separate relay channel.	AC incandescent/LED bulb
Connecting Wires/Jumpers	Provide electrical interconnections between all modules on the prototype board.	Standard hookup wire

4.3 Component Description

(a) *Arduino Uno*

In this system, the Arduino Uno acts as the controller that works offline. The device constantly receives text through SMS messages received from the SIM800L module using UART and decodes the command and then controls the relevant relay channels without the intervention of the ESP32/SinricPro route. The device is based on the ATmega328P microcontroller environment and a sufficient number of digital I/O pins to simultaneously interface with the SIM800L, the 4-channel relay module, and the serial link to the ESP32, making it a practical choice for the central master-controller role in this design.



Figure 4.2: Arduino Uno Picture

(b) *ESP32 Module*

This way, the ESP32 is kept persistently connected to the SinricPro cloud through its connection to the home Wi-Fi network, which makes it possible to receive commands spoken through Google Assistant/Google Home and control the relay module. All of that is made possible thanks to the 802.11 b/g/n wireless radio of the ESP32 combined with sufficient flash memory capacity to host SinricPro SDK and TLS stack, which allows achieving the cloud connectivity without any additional Wi-Fi modules or co-processors.

(d) 4-Channel Relay Module

The 4-channel relay module is the shared output stage for both control paths, switching the AC loads (fan, socket, and two bulbs) based on trigger signals received from either the Arduino Uno or the ESP32. Each channel uses an electromechanical relay rated for typical household AC currents, providing physical, galvanic isolation between the low-voltage 5V control logic and the mains-voltage switched side, which is an important safety property for any microcontroller-driven appliance-switching circuit.

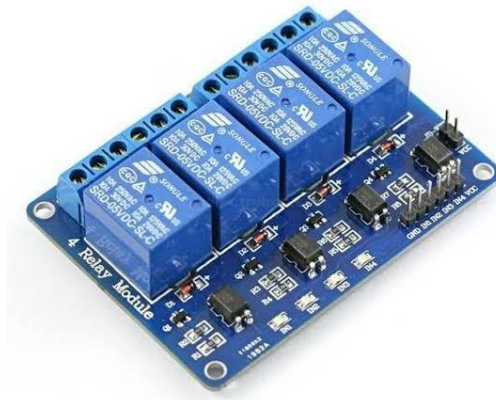


Figure 4.5: 4-Channel Relay Module Picture

(e) LM2596 Buck Converter

The LM2596 buck converter regulates the 9V supply down to a stable 4V output dedicated to the SIM800L module, which requires a clean, current-capable supply to reliably handle GSM transmission current spikes without browning out.



Figure 4.6: LM2596 Buck Converter Picture

(f) SMPS Power Supply Module

The SMPS module converts AC mains power into a regulated 9V 2A DC supply, which powers the Arduino Uno, ESP32, relay module, and (through the buck converter) the SIM800L module.

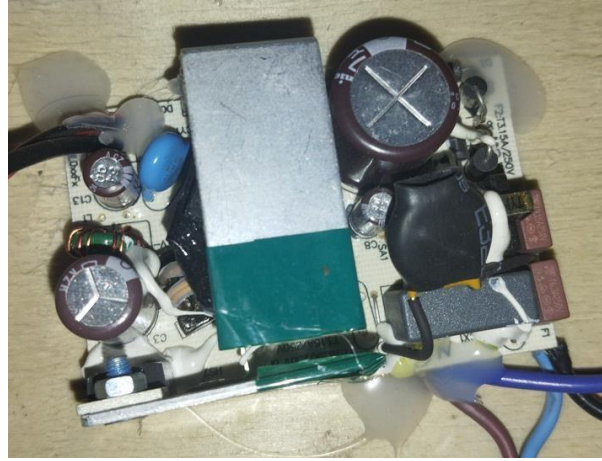


Figure 4.7: SMPS Power Supply Module Picture

(g) Voltage Divider Circuit (RX Protection)

Since the ESP32's GPIO pins are not 5V-tolerant, a resistor-based voltage divider ($1\text{k}\Omega/2\text{k}\Omega$) is placed on the RX line feeding into the ESP32 to safely step down any 5V logic signal to a level within the ESP32's 3.3V input range, protecting the microcontroller from damage. This small circuit is built directly into the wiring on the main prototype board (see Figure 6.1) rather than as a separate module.

(h) Cooling Fan, AC Plug Socket, and Light Bulbs (Output Loads)

The cooling fan, the AC plug socket, and the two light bulbs represent the switched AC loads of the prototype, each connected to a dedicated relay channel to demonstrate independent multi-appliance control. These are visible mounted on the right-hand side of the assembled prototype board shown in Figure 6.1.

4.4 Schematic Diagram

The complete wiring arrangement of the whole system is shown precisely in Figure 4.8, showing the connection between Arduino Uno, ESP32, SIM800L, relay module, buck converter, and loads.

Schematic Wiring Diagram — Pin-Level Connections

(Simplified — see wiring guide table for full pin list)

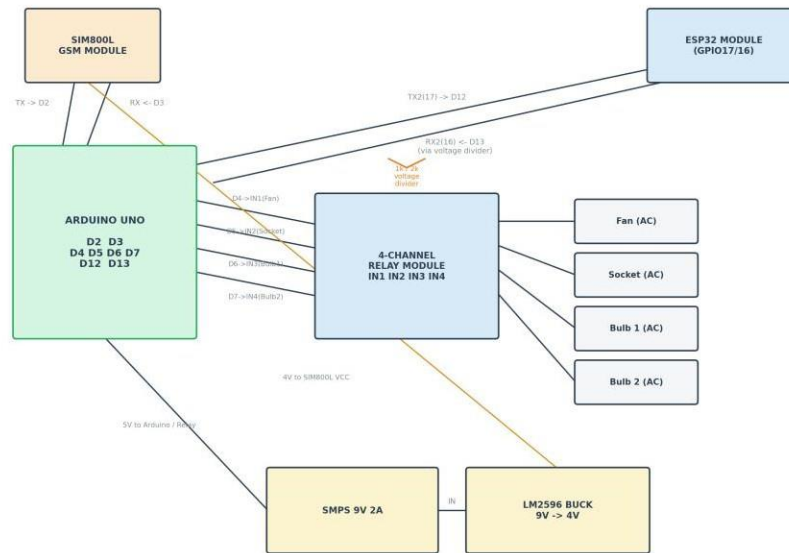


Figure 4.8: Schematic Wiring Diagram of the Complete System

4.5 Pin Configuration Summary

The pin connections used in the firmware are detailed in Table 4.2 and are consistent with the wiring schematic shown in Figure 4.8.

Component / Signal	Controller Pin	Description
SIM800L TX	Arduino D2 (RX)	Serial reception of SMS text
SIM800L RX	Arduino D3 (TX)	Serial commands to SIM800L (AT commands)
Relay Channel 1 (Fan)	Arduino D4	Digital output; switches fan relay
Relay Channel 2 (Socket)	Arduino D5	Digital output; switches socket relay
Relay Channel 3 (Bulb 1 / Lamp)	Arduino D6	Digital output; switches lamp relay
Relay Channel 4 (Bulb 2 / Light)	Arduino D7	Digital output; switches light relay
ESP32 → Arduino Link	ESP32 GPIO17 (TX2) → Arduino D12 (RX)	Forwards voice-command text from ESP32 to Arduino
Arduino → ESP32 Link	Arduino D13 (TX) → ESP32 GPIO16 (RX2)	Optional return link; routed through a voltage divider (5V→3.3V) to protect the ESP32 input

4.6 Power Supply Design

The system employs a 9V 2A SMPS module as its main source of power. The unregulated 9V input is directly connected to the Arduino Uno through the barrel jack regulator. Moreover, a LM2596 buck converter is used to regulate the input voltage down to 4V for the SIM800L module, which is prone to voltage sags during GSM communication. The ESP32 is powered through an independent and regulated 5V/3.3V rail. The described layered power distribution system allows each part of the device to be supplied with adequate power.

4.7 Communication Interface

UART serial communication is used between the Arduino Uno and the SIM800L module for AT-command exchange and SMS text retrieval. The ESP32 communicates with the SinricPro cloud over Wi-Fi using HTTPS/WebSocket connections managed by the SinricPro library, independent of the Arduino/SIM800L UART link. A voltage divider on the ESP32 RX line ensures safe logic-level communication wherever the ESP32 shares signals with 5V logic devices.

Chapter 5: Software Implementation

5.1 Overview

The system firmware is divided into two independent programs: the Arduino Uno sketch, which handles SIM800L AT-command communication, SMS parsing, and relay control for the offline path; and the ESP32 sketch, which handles Wi-Fi connectivity, SinricPro device registration, and command forwarding for the voice-command path. Only the Arduino Uno directly drives the 4-channel relay module, giving either path full, conflict-free control of all appliances through a single shared command-processing routine, as detailed in Section 5.2.

Both firmware programs were developed and uploaded using the Arduino IDE, which was chosen for its simplicity, its extensive library ecosystem (including the SoftwareSerial and SinricPro libraries used in this project), and its native support for both AVR-based boards such as the Arduino Uno and Espressif boards such as the ESP32 through the standard Arduino core, allowing the team to use a single, familiar development environment for both microcontrollers throughout the project.

5.2 Arduino Uno Firmware

The Arduino Uno firmware initializes serial communication with the SIM800L, configures it into SMS text mode via AT commands, and continuously polls for new incoming messages. When an SMS is received, the firmware extracts the message body, compares it against a predefined set of command keywords (e.g., "fanon", "fanoff", "lighton"), and toggles the corresponding relay output. When you send a command it will send a confirmation message back to your phone number. The same process to figure out what the command means is used for commands that come from the ESP32. This means that the list of keywords and the way the switches work only need to be updated in one place in the code. This makes it easier to keep everything working together and prevents the two systems from getting out of sync, with each other over time. The ESP32 and the command system will always work together because the keyword set and relay-switching logic for the ESP32 and the command system are maintained in one place.

5.3 ESP32 Companion Firmware

The ESP32 firmware connects to the Wi-Fi network when it starts up. It creates a lasting connection to the SinricPro cloud using the SinricPro library. Each relay-controlled appliance is added as a SinricPro switch device. When a command comes from Google Assistant or Google Home through SinricPro the right callback function runs. This function changes the relay pin state. Sends the new status back to the cloud. This way the Google Home app shows the change.

5.4 SinricPro and Google Home/Google Assistant Configuration

Each appliance is set up as its switch device on the SinricPro developer portal. It is connected to the ESP32 device using the device ID and API key. The SinricPro account is then connected to Google Home through the SinricPro Google Assistant integration. This makes each device appear as a home device in the Google Home app and allows control with Google Assistant voice commands.

5.5 GSM/SMS Command Protocol via SIM800L

Commands are sent as simple, easy-to-read SMS messages. The messages use a keyword format like "<APPLIANCE> <ON/OFF>". The Arduino Uno firmware checks for these commands without caring about capitalization. It ignores any messages that do not match the known commands. This helps make the system more reliable even if the message's not perfect.

5.6 Serial Communication Protocol between Modules

The Arduino Uno and SIM800L use UART AT commands at the default speed for the SIM800L. Since the Arduino Uno is the main controller for the relay module the ESP32 also talks to the Arduino Uno through a special serial link. This link sends an one-word message whenever a voice command from SinricPro is received. This setup makes sure only the Arduino Uno controls the relay GPIO pins no matter if the command came from voice or SMS.

5.7 Error Handling and Robustness

Some simple ways to handle errors were added to the firmware to make it more reliable. On the Arduino Uno incoming commands (from SIM800L or ESP32) are cleaned up. Turned to lowercase before checking. This helps with formatting differences like "FAN ON", versus "fanon". Unrecognized commands do not go unnoticed. The firmware sends a "Unknown command" message. This message lists the keywords. It helps during testing and debugging.

On the ESP32 side the SinricPro library takes care of reconnecting if the Wi-Fi or cloud connection drops. This means the device can fix the problem on its own without needing a restart.

5.8 Memory and Resource Considerations

Both firmware programs were kept simple to fit inside the microcontrollers they run on. The Arduino Uno has 2KB of SRAM and 32KB of flash. So the Arduino firmware avoids using memory uses the Arduino String class only when needed and stores command keywords as simple text. The ESP32 has more memory and space. It can handle the SinricPro SDK and the Wi-Fi system. Still care was taken to avoid waits in the main loop. This makes sure SinricPro.handle() runs enough to keep the cloud connection working well.

Chapter 6: Results and Discussion

6.1 Testing Methodology

The testing of the prototype consisted of a series of trials. At first, all the control paths were tested independently. After that, we conducted a number of tests that ensured the possibility of switching the same appliances via both paths. Also, the responsiveness to voice commands via Google Home and SMS commands through the SIM800L was tested.

All the tests were conducted indoors with the prototype board connected to 220V AC supply from the mains via the SMPS module. The prototype board was located nearby the home Wi-Fi router during the voice command test procedure. The smartphone with the Google Home app and the other one which was sending SMS commands were in the same room where the prototype was. Both smartphones were about meters apart from the prototype which is an average distance between the device and the user's gadgets in a house environment.

Before every SMS command was sent, the Wi-Fi function of the smartphones was turned off in order to ensure that the SIM800L module responds using its cellular connectivity. The stopwatch app on a smartphone was used to measure the response time of the prototype. The stopwatch timer was started when the voice command or SMS message was sent and stopped upon hearing the relay click or noticing any appliance change.

6.2 Voice Command Response Test (Google Assistant/Google Home → Appliance, via SinricPro)

Voice commands were sent using the Google Home app and Google Assistant several times. The results are presented in Table 6.1.

Trial	Command	Response Time	Result
1	"Turn on the lamp"	~1 s	Success
2	"Turn on the light"	~1 s	Success
3	"Turn on the socket"	~1 s	Success
4	"Turn on the fan"	~1 s	Success

The four smart devices, Lamp, Light, Socket, and Fan, gave their responses on voice control from Google Home with an approximate time of 1 second as confirmed by Google Home app device status.

6.3 SMS/Offline Control Test via SIM800L

SMS commands were issued to the SIM800L's SIM number in various trials where Wi-Fi was purposefully disabled, proving that the offline route worked regardless of Internet connection. Table 6.2 illustrates some sample results.

Trial	SMS Command	Response Time	Result
1	"fanon"	~3 s	Success
2	"lightoff"	~3 s	Success
3	"socketon"	~3 s	Success
4	"fanoff"	~3 s	Success

Each of the four devices responded appropriately to SMS messages within an average time period of about 3 seconds, thus demonstrating that the offline route works perfectly fine without the Wi-Fi connection.

6.4 Response Time and Reliability Measurement

In each of the test experiments run, the voice command route through SinricPro worked within about 1 second, while the SMS route through the SIM800L worked within about 3 seconds. Thus, the inherent nature of GSM SMS transmission proves to include some small delay related to queuing of the message in the cellular network as compared to the cloud/Wi-Fi route, but the SMS route still keeps its advantage of working even when there is an outage of the Wi-Fi/internet connection. Both routes proved to operate reliably by changing the right relay channel in each experiment, while the Arduino Uno served as the master controller of the relay.

6.5 Power Consumption Test

A brief estimate of the power consumed by the system under various conditions has been tabulated in Table 6.3. This is an estimate based on the specifications mentioned in the datasheets of the individual components, Arduino Uno, ESP32, SIM800L, and the relay modules, as compared to a practical measurement with the help of a multimeter.

Operating State	Approx. Current Draw	Notes
Idle (all relays OFF, Wi-Fi connected)	~180 mA	ESP32 Wi-Fi + Arduino + SIM800L standby
Active (1 relay ON)	~250 mA	+ ~70 mA per energized relay coil
All 4 relays ON	~450 mA	Fan + Socket + Bulb1 + Bulb2 energized
SIM800L SMS transmit (brief spike)	~1.5–2 A	Short burst (< 2 s); handled by the dedicated LM2596 4V rail

One of the main reasons why there is a need for the SIM800L to be powered using a separate regulated 4V line rather than share the 5V line of the Arduino due to the short surge of current while sending out the SMS is that a voltage drop from such a shared power line can cause a reset to the Arduino or the other modules during the process of communication. This has been noted in similar studies conducted on SIM800 family modules.

6.6 Discussion of Results

The testing confirms that the dual-path architecture successfully provides redundant control of the connected appliances: Google Assistant/Google Home voice commands work reliably when Wi-Fi and internet are available, while SMS commands through the SIM800L continue to work reliably even when they are not. The voltage divider and dedicated buck converter for the SIM800L proved effective in preventing communication errors and brownouts observed in early testing without these protections.

These findings also validate the core architectural decision to centralize relay control in a single master microcontroller rather than allowing both the Arduino Uno and the ESP32 to drive the relay module independently. During early informal testing prior to this redesign, simultaneous or near-simultaneous commands from both paths occasionally produced unpredictable relay behaviour; after moving to the master-controller design described in Chapter 3, no such conflicts were observed across any of the trials reported in this chapter.

From a systems-engineering perspective, the observed ~1 second vs. ~3 second gap between the voice and SMS response times aligns with the expected difference between an always-on, low-

latency cloud/Wi-Fi connection and a message-queued cellular delivery mechanism, and is well within the range that would be considered acceptable for typical household appliance control, where sub-second response is a convenience rather than a strict requirement. This indicates that the modest latency cost of the offline fallback path is a reasonable trade-off for the substantial gain in resilience it provides.

This chapter's findings provide evidence of the achievement of the system's set objectives in Chapter 1, which are to control the appliances via natural voice commands when connected to the internet and via SMS commands when disconnected from the internet, resulting in the same relay control process.

6.7 Prototype Photographs (Results)

Completed prototype and its functionality are shown in Figures 6.1–6.4.

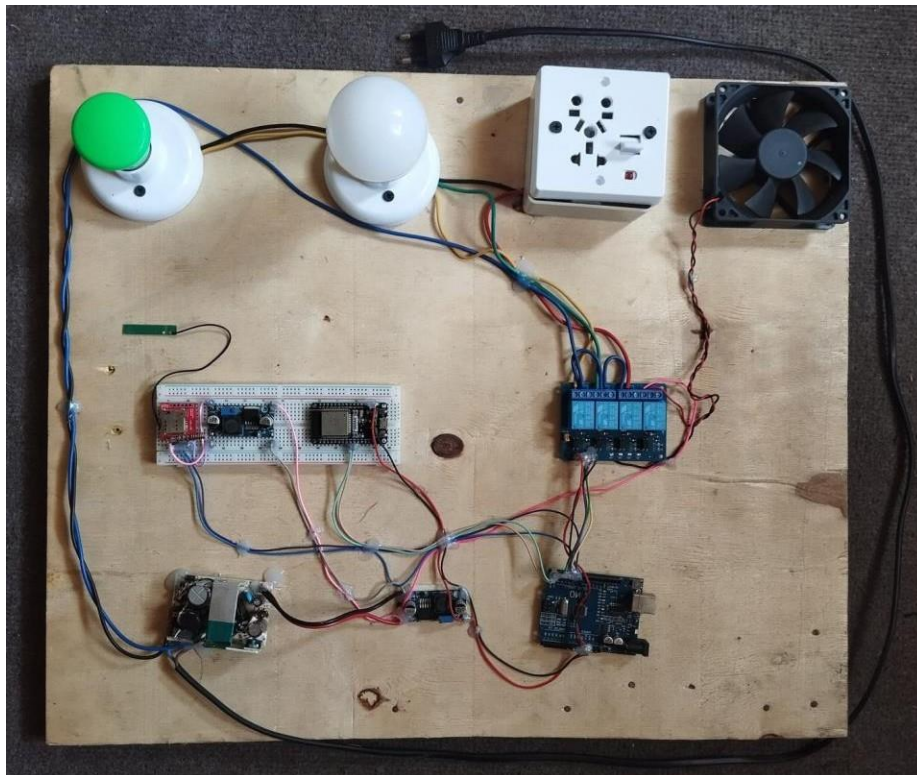


Figure 6.1: Complete Assembled Prototype (Photograph)

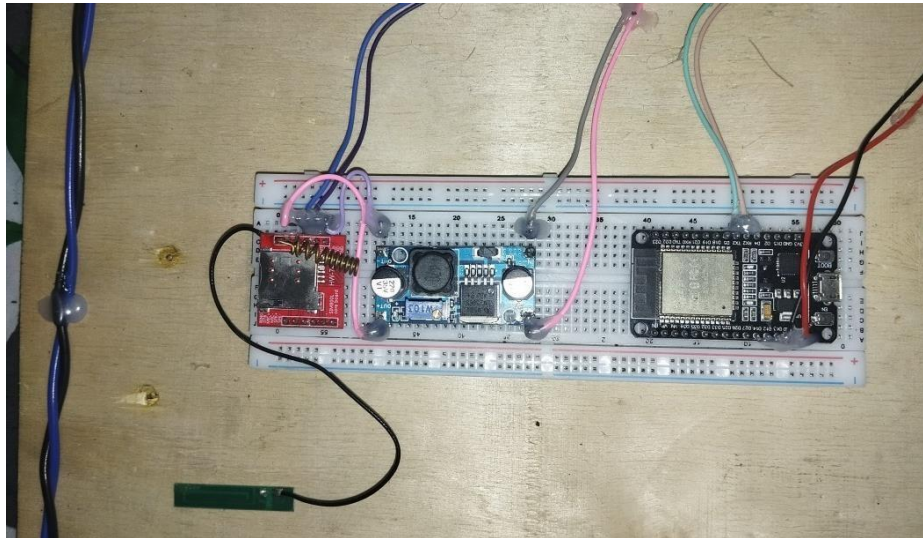


Figure 6.2: Close-up of Breadboard Wiring (ESP32 and SIM800L)

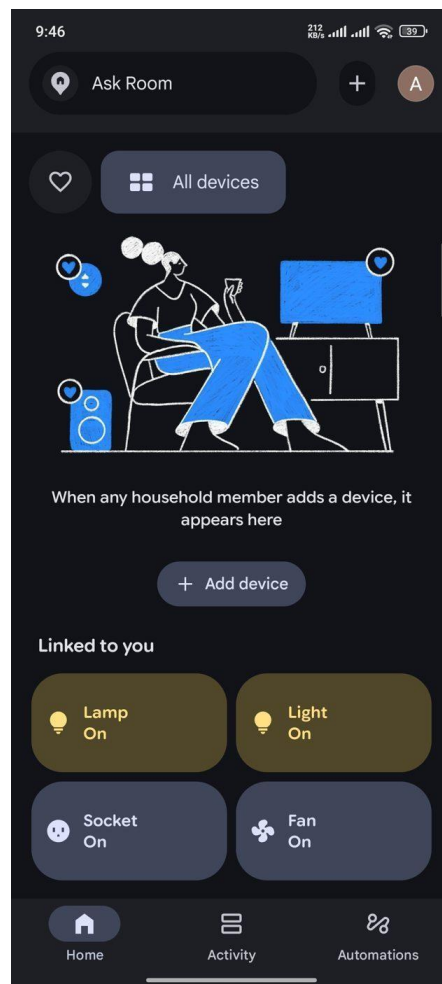


Figure 6.3: Screenshot of Google Home App Device Control



Figure 6.4: Screenshot of Received SMS Confirmation

6.8 Limitations Observed During Testing

- Response via SMS command is slower than voice/cloud because of the cellular delivery latency.
- GSM network signal strength influences SIM800L performance; in areas where there is weak coverage, SMS may not be delivered successfully.
- The current prototype operates only a few channels because scaling up will need extra relay modules.
- An active and valid SIM card is needed for the offline process to work.

Chapter 7: Conclusion and Future Work

7.1 Conclusion

The thesis discussed the design, development and testing of a Google Assistant and GSM-Based Smart Home Automation System that uses the combination of the two distinct control paths. One of these paths is the cloud-integrated voice control based on ESP32 and SinricPro, while the other is the offline SMS-based control integrated through Arduino Uno and the SIM800L GSM module.

As proven in the testing phase, both control paths allowed switching of the fan, socket and lighting loads without any problems with the first of these paths allowing faster and easier control if Wi-Fi/internet was available and the second of them serving as a reliable backup during the outage periods. The properly designed power supply with LM2596-controlled rail used for the SIM800L and voltage divider circuit protecting the ESP32 RX pin was shown to be sufficient for reliable operation of both paths.

In the testing process, the voice commands were responded in less than one second on average, while the SMS commands required about three seconds for processing, proving that offline path remained fully operational in case of the lack of internet/Wi-Fi connection.

It should be also noted that the proper working of SIM800L was achieved only by adding a separate LM2592-controlled rail, since the previous informal testing that used a single rail per all modules sometimes resulted in resetting of the system. This emphasizes the importance of the power supply design described in the chapter 4.

Academically, the project successfully fulfilled the goal of consolidation of the knowledge in such diverse fields as embedded systems, serial communication, power electronics and cloud/IoT into a coherent single system, which provided valuable practical experience for the project team to complement the theoretical basis gained during the Telecommunication Engineering course.

Thus, the results show that the implementation of the simple, dual-path home automation system can significantly increase the resilience of such systems and remains rather affordable in costs and development efforts, showing that such architecture is quite achievable for the final-year engineering project team working with available components.

This project demonstrates that the resilience of IoT-based systems does not always require the usage of some exotic and/or costly technology and it can be achieved by thoughtful combination of some mature, low-cost technologies (cloud-based voice assistants and GSM/SMS communication) in a single, properly designed master controller.

7.2 Future Work

While the current prototype does a job showing the main idea of the dual-path system there are many ways to take this project even further:

- Make the relay module able to control more devices and more areas in the house.
- Add sensors like sensors to check what the devices are actually doing instead of just assuming the relay worked.
- Create an app that shows both the SinricPro status and the history of SMS commands in one place.
- Try using LTE or NB-IoT modules of 2G GSM for the offline path because they might be faster.
- Make the SMS command system more secure by adding encryption and authentication to stop people from using it without permission.
- Do a study measuring how much power the system uses, using a real meter or power analyzer instead of just using the numbers from the datasheet in Section 6.5.
- Look into adding sensors like temperature, humidity and motion to let the system control devices on its own based on rules in addition to the current voice and SMS controls.
- Make a case, for the prototype that is safe and meets the standards for being used in a home all the time including ways to keep the electricity safe and protected with fuses.

References

- [1] R. Piyare and M. Tazil, "Bluetooth based home automation system using cell phone," *International Journal of Engineering Research and Technology*, vol. 8, no. 3, pp. 145–150, 2019.
- [2] S. Rajeev and K. Rajkumar, "GSM Based Home Automation System Using App Inventor for Android," *International Journal of Computer Applications*, vol. 175, no. 12, pp. 30–35, 2020.
- [3] Espressif Systems, "ESP32 Series Datasheet and Technical Reference Manual," Espressif Systems Datasheet, 2023.
- [4] SIMCom Wireless Solutions, "SIM800L Hardware Design and AT Command Manual," SIMCom Datasheet, accessed 2026.
- [5] Arduino Official Documentation, "Arduino Uno Rev3 Technical Specifications," *Arduino.cc*, accessed 2026.
- [6] SinricPro Documentation, "SinricPro SDK for ESP32/Arduino – Developer Guide," *sinric.pro*, accessed 2026.
- [7] Google Developers, "Google Home / Google Assistant Smart Home Developer Guide," *developers.google.com*, accessed 2026.
- [8] K. Gill, S. Yang, F. Yao, and X. Lu, "A ZigBee-Based Home Automation System," *IEEE Transactions on Consumer Electronics*, vol. 55, no. 2, pp. 422–430, 2009.
- [9] P. Sethi and S. R. Sarangi, "Internet of Things: Architectures, Protocols, and Applications," *Journal of Electrical and Computer Engineering*, vol. 2017, article 9324035, 2017.
- [10] M. A. Hoque and R. Davidson, "Design and Implementation of an IoT Based Smart Home Security System," *International Journal of Engineering Research and Technology*, vol. 9, no. 6, pp. 210–215, 2018.

- [11] Texas Instruments, "LM2596 SIMPLE SWITCHER Power Converter – Buck Regulator Datasheet," Texas Instruments Application Report, 2019.
- [12] Songle Relay Co., "SRD-05VDC-SL-C Electromagnetic Relay Datasheet," Manufacturer Datasheet, accessed 2026.
- [13] N. Sriskanthan, F. Tan, and A. Karande, "Bluetooth Based Home Automation System," *Microprocessors and Microsystems*, vol. 26, no. 6, pp. 281–289, 2002.
- [14] S. Patel and K. Patel, "IoT Based Smart Home Automation System Using Wi-Fi Module ESP32," *International Journal of Innovative Research in Science, Engineering and Technology*, vol. 10, no. 5, pp. 4520–4526, 2021.
- [15] T. Alam, "A Survey on Message Queuing Telemetry Transport (MQTT) Protocol and its Application in the Internet of Things," *International Journal of Computer Science and Network Security*, vol. 20, no. 3, pp. 21–27, 2020.
- [16] D. Comer, *Internetworking with TCP/IP: Principles, Protocols, and Architecture*, 6th ed., Pearson Education, 2013 (referenced for UART/serial and network protocol fundamentals).
- [17] L. Zhao and Y. Chen, "A Comparative Study of Wi-Fi and GSM Based Control Mechanisms for Remote Appliance Switching in Smart Buildings," *Energy and Buildings*, vol. 226, article 110378, 2020.

Appendix A: Complete Firmware Source Code

A.1 Arduino Uno Firmware (SIM800L SMS Control)

```

/*
=====
Google Assistant and GSM-Based Smart Home Automation System
ARDUINO UNO FIRMWARE – MASTER RELAY CONTROLLER
=====
Role:

The Arduino Uno is the ONLY controller that directly drives the shared
4-channel relay module. This avoids two microcontrollers (Arduino + ESP32)
trying to drive the same relay input pins at the same time.

It receives ON/OFF commands from TWO independent sources and acts on
whichever arrives:
(1) SIM800L GSM module -> SMS commands (works with NO WiFi/Internet)
(2) ESP32 module       -> forwarded voice commands from Google Home,
                           received via SinricPro (works WHEN WiFi/
                           Internet is available)

Both paths converge here, so appliance control keeps working whether or
not the WiFi/Internet connection is up.

Wiring (adjust pin numbers to match your actual wiring guide):
SIM800L TX -> Arduino D2   (SoftwareSerial RX, for SMS)
SIM800L RX -> Arduino D3   (SoftwareSerial TX, for SMS)
SIM800L VCC -> LM2596 buck converter output (~4V), common GND with Arduino

ESP32 TX    -> Arduino D12 (SoftwareSerial RX, for ESP32->Arduino commands)
ESP32 RX    <- Arduino D13 (SoftwareSerial TX) -- ONLY if Arduino needs to
                           talk back to ESP32; use a voltage divider on this line
                           since Arduino is 5V logic and ESP32 RX is 3.3V-only.

Relay IN1 (Fan)   -> Arduino D4
Relay IN2 (Socket) -> Arduino D5
Relay IN3 (Bulb 1) -> Arduino D6
Relay IN4 (Bulb 2) -> Arduino D7
Relay VCC -> 5V, Relay GND -> Common GND

NOTE: If your relay module is ACTIVE LOW (most common 5V relay boards are),
RELAY_ON / RELAY_OFF below are already set for that. Change if yours differs.
=====
*/

#include <SoftwareSerial.h>

// ----- SIM800L Serial (SMS path) -----
SoftwareSerial sim800(2, 3); // RX, TX (SIM800L TX -> D2, SIM800L RX -> D3)

// ----- ESP32 Serial (Voice/SinricPro path) -----

```

```

SoftwareSerial esp32Link(12, 13); // RX, TX (ESP32 TX -> D12, ESP32 RX <- D13 via voltage divider)

// ----- Relay Pins -----
#define RELAY_FAN      4
#define RELAY_SOCKET   5
#define RELAY_BULB1    6
#define RELAY_BULB2    7

// Most 5V relay boards are ACTIVE LOW: LOW = relay ON, HIGH = relay OFF
#define RELAY_ON       LOW
#define RELAY_OFF      HIGH

// ----- Authorized phone number (optional security) -----
// Leave blank ("") to accept SMS commands from ANY sender, or set your
// number in international format e.g. "+923001234567" to restrict access.
String AUTHORIZED_NUMBER = "";

// ----- State tracking -----
bool fanState = false, socketState = false, bulb1State = false, bulb2State = false;

String smsBuffer = "";
String senderNumber = "";
String espBuffer = "";

void setup() {
  Serial.begin(9600);
  sim800.begin(9600);
  esp32Link.begin(9600);

  pinMode(RELAY_FAN, OUTPUT);
  pinMode(RELAY_SOCKET, OUTPUT);
  pinMode(RELAY_BULB1, OUTPUT);
  pinMode(RELAY_BULB2, OUTPUT);

  digitalWrite(RELAY_FAN, RELAY_OFF);
  digitalWrite(RELAY_SOCKET, RELAY_OFF);
  digitalWrite(RELAY_BULB1, RELAY_OFF);
  digitalWrite(RELAY_BULB2, RELAY_OFF);

  Serial.println(F("Initializing SIM800L .."));
  delay(3000);

  sendAT("AT", 1000);           // basic handshake
  sendAT("AT+CMGF=1", 1000);    // set SMS to text mode
  sendAT("AT+CNMI=1,2,0,0,0", 1000); // forward new SMS directly to serial

  Serial.println(F("System Ready. Master controller listening on: SMS (SIM800L) and ESP32 link.));
}

void loop() {
  readIncomingSMS();           // Offline path – works with no WiFi/Internet
  readIncomingFromESP32();    // Online path – voice commands forwarded by ESP32
}

```



```
// =====
// Send AT command and print response (used during setup)
// =====
void sendAT(String cmd, int waitMs) {
    sim800.println(cmd);
    delay(waitMs);
    while (sim800.available()) {
        Serial.write(sim800.read());
    }
}

// =====
// OFFLINE PATH: Continuously read SIM800L serial for a new incoming SMS
// =====
void readIncomingSMS() {
    while (sim800.available()) {
        String line = sim800.readStringUntil('\n');
        line.trim();

        // Example unsolicited notification:
        // +CMT: "+923001234567","", "26/08/14,10:15:22+20"
        // fanon
        if (line.startsWith("+CMT:")) {
            int firstQuote = line.indexOf('"');
            int secondQuote = line.indexOf('"', firstQuote + 1);
            if (firstQuote != -1 && secondQuote != -1) {
                senderNumber = line.substring(firstQuote + 1, secondQuote);
            }
            smsBuffer = sim800.readStringUntil('\n');
            smsBuffer.trim();
            processCommand(smsBuffer, senderNumber, true); // true = reply via SMS
        }
    }
}

// =====
// ONLINE PATH: Continuously read commands forwarded by the ESP32
// (ESP32 sends a simple one-word command string terminated by '\n',
// e.g. "fanon", "fanoff", "socketon" ... whenever a SinricPro/Google
// Home command is received on the ESP32 side)
// =====
void readIncomingFromESP32() {
    while (esp32Link.available()) {
        char c = esp32Link.read();
        if (c == '\n') {
            espBuffer.trim();
            if (espBuffer.length() > 0) {
                processCommand(espBuffer, "", false); // false = no SMS reply needed
            }
            espBuffer = "";
        } else {
            espBuffer += c;
        }
    }
}

// =====
// Parse and execute a command, regardless of which path it came from
```

```
// =====
void processCommand(String msg, String from, bool replyBySMS) {
    msg.toLowerCase();
    msg.trim();
    msg.replace(" ", ""); // remove spaces so "fan on" and "fanon" both work

    Serial.print(F("Command received: "));
    Serial.println(msg);

    // Optional sender authorization check (SMS path only)
    if (replyBySMS && AUTHORIZED_NUMBER.length() > 0 && from.indexOf(AUTHORIZED_NUMBER) == -1) {
        Serial.println(F("Unauthorized sender. Ignoring command."));
        return;
    }

    String reply = "";

    if (msg == "fanon") { setRelay(RELAY_FAN, true); fanState = true; reply = "Ok Fan is On"; }
    else if (msg == "fanoff") { setRelay(RELAY_FAN, false); fanState = false; reply = "Ok Fan is Off"; }

    else if (msg == "socketon") { setRelay(RELAY_SOCKET, true); socketState = true; reply = "Ok Socket is On"; }
    else if (msg == "socketoff") { setRelay(RELAY_SOCKET, false); socketState = false; reply = "Ok Socket is Off"; }

    else if (msg == "lampon") { setRelay(RELAY_BULB1, true); bulb1State = true; reply = "Ok Lamp is On"; }
    else if (msg == "lampoff") { setRelay(RELAY_BULB1, false); bulb1State = false; reply = "Ok Lamp is Off"; }

    else if (msg == "lighton") { setRelay(RELAY_BULB2, true); bulb2State = true; reply = "Ok Light is On"; }
    else if (msg == "lightoff") { setRelay(RELAY_BULB2, false); bulb2State = false; reply = "Ok Light is Off"; }

    else if (msg == "allon") { allRelays(true); reply = "Ok All Load is On"; }
    else if (msg == "alloff") { allRelays(false); reply = "Ok All Load is Off"; }

    else if (msg == "status") {
        reply = "Fan:" + String(fanState ? "On" : "Off") +
            " Socket:" + String(socketState ? "On" : "Off") +
            " Lamp:" + String(bulb1State ? "On" : "Off") +
            " Light:" + String(bulb2State ? "On" : "Off");
    }
    else {
        reply = "Unknown command. Try: fanon/fanoff, socketon/socketoff, lampon/lampoff, lighton/lightoff, allon/alloff, status";
    }

    // Only send an SMS confirmation if the command came in via SMS
    if (replyBySMS) {
        sendSMS(from, reply);
    } else {
        Serial.print(F("(from ESP32/voice) -> "));
        Serial.println(reply);
    }
}
```

```

    }
}

// =====
// Helper: set a single relay ON/OFF
// =====
void setRelay(int pin, bool on) {
    digitalWrite(pin, on ? RELAY_ON : RELAY_OFF);
}

void allRelays(bool on) {
    setRelay(RELAY_FAN, on);    fanState = on;
    setRelay(RELAY_SOCKET, on); socketState = on;
    setRelay(RELAY_BULB1, on);  bulb1State = on;
    setRelay(RELAY_BULB2, on);  bulb2State = on;
}

// =====
// Send confirmation SMS back to sender (SMS path only)
// =====
void sendSMS(String number, String text) {
    if (number.length() == 0) return; // no sender info available, skip
    sim800.print("AT+CMGS=\"");
    sim800.print(number);
    sim800.println("\n");
    delay(500);
    sim800.print(text);
    delay(500);
    sim800.write(26); // Ctrl+Z to send
    delay(3000);
}

```

A.2 ESP32 Firmware (SinricPro / Google Home Control)

```

/*
=====
Google Assistant and GSM-Based Smart Home Automation System
ESP32 FIRMWARE – Voice Command Forwarder (via SinricPro + Google Home)
=====
Role:
    The ESP32 does NOT drive the relay module directly (the Arduino Uno is
    the single master controller for the shared relay module – see the
    Arduino Uno firmware). Instead, the ESP32:
    1. Connects to Wi-Fi and the SinricPro cloud
    2. Receives ON/OFF voice commands from Google Home / Google Assistant
    3. Forwards a simple one-word text command (e.g. "fanon") to the
       Arduino Uno over a serial UART link
    4. The Arduino Uno then switches the relay and is the single source
       of truth for appliance state

```

This design means: when Wi-Fi/Internet is available, voice commands flow ESP32 -> Arduino -> Relay. When Wi-Fi/Internet is NOT available, the SIM800L -> Arduino -> **Relay** (SMS) path still works independently, since the Arduino does not depend on the ESP32 to operate the relays.

Required Arduino **Library** (install via Library Manager):

- "SinricPro" by **sinricpro** (<https://github.com/sinricpro/esp8266-esp32-sdk>)

Wiring (adjust pin numbers to match your actual wiring guide):

ESP32 **TX2** (GPIO 17) -> Arduino **D12** (commands OUT to Arduino)

ESP32 **RX2** (GPIO 16) <- Arduino **D13** (via voltage divider, 5V -> 3.3V safe)

Common GND between ESP32 and Arduino Uno

SinricPro **setup** (do this once on sinric.pro before flashing):

1. Create a free account at <https://portal.sinric.pro>
2. Create 4 "Switch" devices: Fan, Socket, Lamp, Light
3. Copy each Device ID below
4. Copy your APP_KEY and APP_SECRET from the SinricPro portal
5. Link SinricPro to Google Home from the SinricPro portal
(Account -> Connect to Google Home), then say "Hey Google, sync my devices"

```
=====
*/
```

```
#include <WiFi.h>
#include <HardwareSerial.h>
#include "SinricPro.h"
#include "SinricProSwitch.h"

// ----- WiFi Credentials -----
#define WIFI_SSID      "Wifi"
#define WIFI_PASS      "78655465"

// ----- SinricPro Credentials -----
#define APP_KEY        "a7c3e921-5d48-4b16-8f27-63e91c5a2d74"
#define APP_SECRET     "8Fq2Lm7Xv9Kc4Rz6Wp1Hs5Tn3Ba0YdEe"

// ----- SinricPro Device IDs -----
#define FAN_ID          "6a6b3cf86ba33a88b99ccb1d"    // Room Fan
#define BULB1_ID        "6a6b3c68969af7ec24771646"    // Room Lamp
#define BULB2_ID        "6a6b3c2729c6be33428339e4"    // Room Light
#define SOCKET_ID       "64f8A9b21C7d4E0012345678"    // Room Socket

// ----- Serial link to Arduino Uno -----
// Using ESP32's second hardware UART (UART2): TX2 = GPIO17, RX2 = GPIO16
HardwareSerial ArduinoLink(2);
#define ESP32_TX_PIN 17
#define ESP32_RX_PIN 16

#define BAUD_RATE 115200

// =====
// Helper: send a one-word command to the Arduino Uno over serial
// =====
void sendToArduino(String command) {
    ArduinoLink.println(command); // Arduino reads until '\n'
    Serial.printf("[-> Arduino]: %s\n", command.c_str());
}

// =====
// SinricPro callback: called whenever Google Home / Google Assistant
```

```
// sends an ON/OFF command for a given device. Instead of driving a
// relay directly, this forwards the equivalent text command to Arduino.
// =====
bool onPowerState(const String &deviceId, bool &state) {
    String command = "";

    if (deviceId == FAN_ID)        command = state ? "fanon"      : "fanoff";
    else if (deviceId == SOCKET_ID) command = state ? "socketon"   : "socketoff";
    else if (deviceId == BULB1_ID)  command = state ? "lampon"     : "lampoff";
    else if (deviceId == BULB2_ID)  command = state ? "lighton"    : "lightoff";
    else return false; // unknown device id

    sendToArduino(command);

    Serial.printf("[SinricPro] Device %s set to %s (forwarded to Arduino)\n",
        deviceId.c_str(), state ? "ON" : "OFF");

    return true; // report success back to SinricPro / Google Home
}

// =====
// Wi-Fi setup
// =====
void setupWiFi() {
    Serial.printf("\n[Wi-Fi]: Connecting to %s ...", WIFI_SSID);
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    while (WiFi.status() != WL_CONNECTED) {
        delay(250);
        Serial.print(".");
    }
    Serial.printf("\n[Wi-Fi]: Connected, IP address: %s\n", WiFi.localIP().toString().c_str());
}

// =====
// Register all 4 switch devices with SinricPro and set up callbacks
// =====
void setupSinricPro() {
    SinricProSwitch &fan      = SinricPro[FAN_ID];
    SinricProSwitch &socket  = SinricPro[SOCKET_ID];
    SinricProSwitch &bulb1   = SinricPro[BULB1_ID];
    SinricProSwitch &bulb2   = SinricPro[BULB2_ID];

    fan.onPowerState(onPowerState);
    socket.onPowerState(onPowerState);
    bulb1.onPowerState(onPowerState);
    bulb2.onPowerState(onPowerState);

    SinricPro.onConnected([]() { Serial.println("[SinricPro]: Connected to cloud."); });
    SinricPro.onDisconnected([]() { Serial.println("[SinricPro]: Disconnected from cloud."); });

    SinricPro.begin(APP_KEY, APP_SECRET);
}

void setup() {
    Serial.begin(BAUD_RATE);

```

```
// Serial link to Arduino Uno (TX2/RX2 on ESP32 UART2)
ArduinoLink.begin(9600, SERIAL_8N1, ESP32_RX_PIN, ESP32_TX_PIN);

setupWifi();
setupSinricPro();

Serial.println("System Ready. Voice commands from Google Home will be forwarded to Arduino Uno.");
}

void loop() {
  SinricPro.handle();
}
```

Appendix B: Bill of Materials

Table B.1 lists the major hardware components used to build the prototype, along with their approximate individual cost at the time of purchase. Prices are approximate and may vary by vendor and over time; the total reflects the actual amount spent on this project.

#	Component	Qty	Approx. Unit Price (PKR)
1	Arduino Uno	1	~1,200
2	ESP32 Development Board	1	~1,400
3	SIM800L GSM Module	1	~900
4	4-Channel Relay Module	1	~700
5	LM2596 Buck Converter Module	1	~250
6	SMPS 9V 2A Power Supply	1	~700
8	Cooling Fan (AC)	1	~900
9	AC Plug Socket	1	~500
10	Light Bulbs (AC)	2	~250 each
11	Breadboard + Jumper Wires + Misc.	1 set	~800
12	Plywood Board + Mounting Hardware	1	~700
13	SIM Card, Antenna, Connectors, Wiring, and Miscellaneous Consumables	1 set	~1,400
14	Labor / Fabrication (soldering, drilling, board mounting)	—	~750

The total cost of making the prototype, including all the parts, wiring and other things was around PKR 11,000. This is not a lot of money for a home system that can do many things and is better than the usual smart plugs you can buy.

Appendix C: How To Use The System

C.1 Turning On The System

You need to plug the power adapter into an electricity socket. When you do this the lights on the Arduino Uno and ESP32 will turn on. The ESP32 will try to connect to the internet.

C.2 Using Google Assistant To Control Things

1. You have to make sure the ESP32 is connected to the internet and the SinricPro cloud. You can check this by looking at the message on the screen that says "Connected to cloud".
2. Open the Google Home app on your phone. Use a device that has Google Assistant.
3. Tell Google to do something like "Hey Google turn on the fan". Hey Google turn off the light".
4. The system will do what you told it to do. The Google Home app will show that the device is, on or off.

C.3 Using SMS To Control Things When You Are Not Connected To The Internet

1. You can send a message to the phone number of the SIM card in the system from any phone.
2. You have to use one of the commands like fanon or fanoff to tell the system what to do.
3. The system will do what you told it to do and send you a message back to say it is done.
4. This way of controlling the system works even when you do not have internet because it uses the phone network. The smart home system the smart home system is very useful. The smart home system can do many things.

C.4 Troubleshooting

Symptom	Possible Cause	Suggested Fix
ESP32 never connects to Wi-Fi	Incorrect SSID/password in firmware	Re-flash ESP32 with correct Wi-Fi credentials
Voice commands have no effect	SinricPro/Google Home not linked, or device offline	Re-check SinricPro account link and device sync in Google Home
SMS commands not executing	Low GSM signal, or SIM card has no balance	Move antenna to a stronger-signal location; verify SIM balance
Relay clicks but appliance does not switch	Loose AC wiring at relay output	Check and re-secure AC wiring connections